

A List of 45+ Mech Interp Project Ideas from Apollo Research's Interpretability Team

by Lee Sharkey, Lucius Bushnaq, Dan Braun, StefanHex, Nicholas Goldowsky-Dill 18th Jul 2024

Why we made this list:

- The interpretability team at Apollo Research wrapped up a few projects recently^[1]. In order to decide what we'd work on next, we generated a lot of different potential projects. Unfortunately, we are computationally bounded agents, so we can't work on every project idea that we were excited about!
- Previous lists of project ideas (such as Neel's collation of [200 Concrete Open Problems in Mechanistic Interpretability](#)^o) have been very useful for people breaking into the field. But for all its merits, that list is now over a year and a half old. Therefore, many project ideas in that list aren't an up-to-date reflection of what some researchers consider the frontiers of mech interp.

We therefore thought it would be helpful to share our list of project ideas!

Comments and caveats:

- Some of these projects are more precisely scoped than others. Some are vague, others are more developed.
- Not every member of the team endorses every project as high priority. Usually more than one team member supports each one, and in many cases most of the team is supportive of someone working on it.

- We associate the person(s) who generated the project idea to each idea.
- We've grouped the project ideas into categories for convenience, but some projects span multiple categories. We don't put a huge amount of weight on this particular categorisation.

We hope some people find this list helpful!

We would love to see people working on these! If any sound interesting to you and you'd like to chat about it, don't hesitate to reach out.

Foundational work on sparse dictionary learning for interpretability

- Transcoder-related project ideas
 - See [\[2406.11944\] Transcoders Find Interpretable LLM Feature Circuits](#))
 - **[Nix] Training and releasing high quality transcoders.**
 - Probably using top k
 - GPT2 is a classic candidate for this. I'd be excited for people to try hard on even smaller models, e.g. GELU 4L
 - **[Nix] Good tooling for using transcoders**
 - Nice programming API to attribute an input to a collection of paths (see [Dunefsky et al](#))
 - Web user interface? Maybe in collaboration with neuronpedia. Would need a gpu server constantly running, but I'm optimistic you could do it with a ~a4000.
 - **[Nix] Further circuit analysis using transcoders.**
 - Take random input sequences, run transcoder attribution on them, examine the output and summarize the findings.
 - High level summary statistics of how much attribution goes through error terms & how many pathways are needed would be valuable
 - Explaining specific behaviors (IOI, greater-than) with high standards for specificity & faithfulness. Might be convoluted if accuracy

- [I could generate more ideas here, feel free to reach out - nix@apolloresearch.ai]
- **[Nix, Lee] Cross layer superposition**
 - Does it happen? Probably, but it would be nice to have specific examples! Look for features with similar decoder vectors, and do exploratory research to figure out what exactly is going on.
 - What precisely does it mean? Answering this question seems likely to shed light on the question of 'What is a feature?'.
- **[Lucius] Improving transcoder architectures**
 - Some MLPs or attention layers may implement a simple linear transformation in addition to actual computation. If we modify our transcoders to include a linear 'bypass' that is not counted in the sparsity penalty, do we improve performance since we are not unduly penalizing these linear transformations that would always be present and active?
 - If we train multiple transcoders in different layers at the same time, can we include a sparsity penalty for their interactions with each other, encouraging a decomposition of the network that leaves us with as few interactions between features as possible to make reverse engineering circuits easier?

• Other

- **[Nix] Idea for improved-logit lens style interpretation of SAE features**
 - Train a high quality SAE at the residual stream pre-unembed. This will require customizing the hyperparameter for this particular SAE, as current sweeps over layers generally have a really bad SAE here (eg [Joseph Bloom's GPT2 SAEs](#)°)
 - Interpret these SAE features, by their effect on classes of logits.
 - Interpret SAE features from earlier layers, in terms of their effect
 - This is basically logit-lens (see [Understanding SAE Features with the Logit Lens](#)°) but in the pre-unembed basis instead of the token basis.
 - Can also look at second order effects (through attention heads with frozen attention patterns), as in [Interpreting the Second-Order Effects of Neurons in CLIP](#)
- **[Nix] Toy model of feature splitting**
 - There are at least two explanations for feature splitting I find plausible:

- Activations exist in higher dimensional manifolds in feature space, feature splitting is a symptom of one higher dimensional mostly-continuous feature being chunked into discrete features at different resolutions.
- There is a finite number of highly-related discrete features that activate on similar (but not identical) inputs and cause similar (but not identical) output actions. These can be summarized as a single feature with reasonable explained variance, but is better summarized as a collection of “split” features.
- I think toy models that make either of these hypotheses more concrete would be valuable. I'd probably start with the second one and find a setting that's analogous to token-in-context features, and train a model to predict different (but highly correlated) bigrams depending on context.
- Could also consider a toy-model of superposition style setup. They tested features with correlated inputs but not correlated outputs.
- **[Dan] Looking for opposing feature directions in SAEs**
 - SAEs are only capable of learning halfspaces (positive activations of some direction above some threshold). But if the ‘true’ underlying feature is a subspace (i.e. both a positive and negative direction), then SAEs will have to learn two, opposite-facing directions to represent this ‘true’ feature. In practice, we notice this happening when we train SAEs to sparsely represent dense, Gaussian point clouds when we use an L_p norm with $p < 1$. Do SAEs learn opposite-facing pairs in language models?
- **[Dan] SAE/Transcoder activation shuffling**
 - There has been no public analysis of the importance of activation shuffling on SAEs. This is valuable information for training setups such as [e2eSAEs](#), which do not shuffle activations during training as they need to pass through the entire context-length activations to subsequent layers. Can you get away with just having a larger effective batch size and higher learning rate? Note that I think this is equally (if not more) important to analyze for transcoders.
- **[Dan] SAE/Transcoder initialization**
 - For e2eSAE training, how much e2e training is needed if initializing SAEs with standard (“local”) training to match performance of training e2e from scratch?

- How much does initializing the encoder to be the transpose of the decoder (as done so [here](#) and [here](#)) help for SAEs and transcoders?
- Find more efficient initializations.
- **[Dan] Make public benchmarks for SAEs and transcoders.**
 - It's very hard for researchers to evaluate the quality of SAEs and transcoders. Having a central place where people can upload their models to and have them be evaluated would be extremely valuable for the community. If interested in helping with this, please reach out to [Neuronpedia](#) which I deem to be a great place to host such a service.
- **[Lee] Mixture of Expert SAEs**
 - Leo Gao has developed fast, sparse, Open Source kernels for SAE training [here](#). This is great! The more efficient we can make SDL the better. But this only speeds up inference of the decoder. I think MOEs may be a way to speed up inference of the encoder.
 - Feature splitting and feature geometry indicate that there is a hierarchical structure to feature-space. MOE-SAEs may be one way to leverage this. They may also have the benefit of letting us study the geometric structure of the features by studying the gate function of the MOE-SAE-encoder. (This suggests an additional question: Can we build SAEs with even more hierarchy? This may let us study feature geometry on an even more abstract level).
 - Rather than training MOE-SAEs from scratch, an initial investigation might be to investigate how well SAEs can be [MOEified](#) post hoc. If they can be, then it's evidence in the direction that MOEs might be reasonable to use during training from scratch.
 - Leaving the potential training benefits aside, we may wish to do lots of inference with SAEs (for instance, during monitoring for the activation of safety-relevant features). It will therefore be important to have efficient ways to do this inference.
- **[Lee] Identify canonical features that emerge in language models**
 - When investigating visual networks, we basically always find certain features. Edge features, for example, appear in both convolutional, transformer-based, and biological networks.
 - When we train SAEs in language models of different size, architecture, training dataset, etc., we might find features that appear again and again.

What are these features?

- And can we use this information to accelerate sparse dictionary learning? For instance, in vision models, we could present the model with inputs consisting only of edges in order to find the 'edge detector' direction. Can we do the same in language models?
- Downscoped: Identify canonical features in early layers of language models
- **[Lee] Studying generalization of SAEs and transcoders.**
 - How well do they generalize to different data distributions?
 - Do some features only activate in long contexts?
 - How easy is it to find adversarial examples for different features and SAEs trained using different methods (e.g. e2e SAEs)?
- **[Lee] How does layer norm affect SAE features before and after?**
 - Alternatively: What do transcoders reveal about how layer norm transforms features?
- **[Lee] Connecting SAE/transcoder features to polytopes**
 - The main downfall of the [polytope lens](#)[°] was that it used clustering methods in order to group polytopes together. This means the components of the explanations they provided were not 'composable'. We want to be able to break down polytopes into components that are composable.
 - Presumably, SAEs and transcoders also capture 'directions' that take similar pathways through the network. The same can be said for polytopes. What is the relationship between features identified using sparse dictionary learning and compositional polytope codes?
 - We could decompose the 'polytope codes' of various data points in a similar way to how we decompose activations. Various methods may be used here, including PCA, ICA, NMF, or SAEs on polytope codes. Note that, since polytope codes at layer L involve concatenating polytope codes of multiple (later) layers together, even PCA on the codes can yield an overcomplete 'basis' at layer L.
- **[Stefan] Verify (SAE) features based on the model weights; show that features are a model-property and not (only) a dataset property.**
 - Details [here](#)[°] (e.g. "given two sets of directions that reconstruct an activation, can you tell which one are the features vs a made-up set of directions?"), one possible methodology described [here](#) (in-progress LASR project)[°].

- **[Stefan] Relationship between Feature Splitting, Feature Completeness, and atomic vs. composite features**
 - Answer whether the specialized features we find in large SAEs “true features” (Anthropic), or whether they represent composites of multiple features. Details [here](#)°.
- **[Lee] Is there structure in feature splitting?**
 - Suppose we have a trained SAE with N features. If we apply e.g. NMF or SAEs to these directions are there directions that explain the structure of the splitting? As in, suppose we have a feature for math and a feature for physics. And suppose these split into (among other things)
 - ‘topology in a math context’
 - ‘topology in a physics context’
 - ‘high dimensions in a math context’
 - ‘high dimensions in a physics context’
 - Is the topology-ifying direction the same for both features? Is the high-dimensionifying direction the same for both features? And if so, why did/didn't the original SAEs find these directions?
- **[Lucius] Understanding the geometry of SAE features**
 - Run PCA on the activations of trained SAEs. How many principal components are small enough to be ablated without hurting model performance much? The more components like this exist, the worse the SAE quality since many of the features are just linear combinations of each other. This makes the PCA spectrum of SAE activations a potential metric for SAE quality.
 - How do LLMs encode digit and number features? Can we guess how models count and perform arithmetic by looking at the geometry of digit embeddings with respect to each other? E.g., if models were performing modular addition with carryover digits to add numbers, we might expect to see the digits embedded in order along a circle, 0,1,2,3,4,5,6,7,8,9, much like the months of the year were found to be embedded in a circle in a recent [paper](#).
 - Does the feature geometry reflect subset/superset relationships? E.g., every bird is an animal, but not every animal is a bird. Every square is a rectangle, but not every rectangle is a square.

- To what extent are SAE features embedded in multiple different subspaces, as opposed to one global space? According to sparse coding theory, one global space of size d can store more features than two subspaces of size $d/2$, but the latter configuration can have more features active at the same time. If there are subspaces, do they line up with semantic relationships? Do they correspond to circuits?
- **[Lucius] Identify better SAE sparsity penalties by reasoning about the distribution of feature activations**
 - In sparse coding, one can derive what prior over encoded variables a particular sparsity penalty [corresponds to](#). E.g. an L1 penalty assumes a Laplacian prior over feature activations, while a $\log(1+a^2)$ would assume a Cauchy prior. Can we figure out what distribution of feature activations over the data we'd expect, and use this to derive a better sparsity penalty that improves SAE quality?
- **[Lucius] Preprocessing activations with the [interaction basis](#) prior to SAE training**
 - Regular SAEs are trained to reconstruct the hidden activations in a neural network layer. However, the size of the hidden activations does not necessarily reflect their importance for computing the final network output. Thus, some small/large discrepancies between the original and reconstructed hidden activations might correspond to large/small discrepancies in the final output of the network. One way this problem might be solved is by training SAEs [end-to-end](#). But another way to solve it might be to preprocess the network activations before applying the SAEs to them. The activations could be rotated and rescaled such that the variance of the hidden activations along any axis is proportional to its importance for computing the final network outputs. The interaction basis is a linear coordinate transformation for the hidden activations of neural networks that attempts to achieve just that. So transforming activations into the interaction basis before applying SAEs to them might yield a Pareto improvement in SAE quality.
- **[Lucius] Using attribution sparsity penalties to improve end-to-end SAEs**
 - For [end-to-end dictionary learning](#), a [sparsity penalty based on attributions](#) might be more appropriate than a sparsity penalty based on dictionary activations: In end-to-end SAEs, the reconstruction loss cares about the final network output, but the sparsity term still cares about the activations in the hidden layer, like a conventional SAE. This is perhaps something of a

mismatch. For example, if a feature is often present in the residual stream, but comparatively rarely used in the computation, the end-to-end SAE will be disinclined to represent it, because it only decreases the reconstruction loss a little, but increases the sparsity loss by a lot. More generally, how large a feature activation is just won't be that great of a correlate for how important it is for reconstructing the output. So if we care about how many features we need per data point to get good output reconstruction, SAEs trained with an attribution sparsity penalty might beat SAEs trained with an activation sparsity penalty.

- The Anthropic proposal for [an attribution sparsity penalty](#) uses attributions of the LLM loss. I suspect this is inappropriate, since the gradient of the LLM loss is zero at optima, meaning feature attributions will be scaled down the better the LLM does on a specific input. Something like an MSE average over attributions to all of the network's output logits might be more appropriate. This is expensive, but an approximation of the average using stochastic sources might suffice. See e.g. Appendix C [here](#) for an introduction to stochastic source methods. In our experiments on the Toy Model of Superposition, a single stochastic source proved to be sufficient, making this potentially no more computationally intensive than the Anthropic proposal.
- In our experiments, training with an attribution sparsity penalty benefited from choosing L_p norm values smaller than 1. We suspect this is because the attribution sparsity term with $L_p=1$ can act a bit like a conventional activation sparsity term with $L_p=2$, since it's a product of an activation and a gradient.

Applied interpretability

- **[Lee] Apply SAEs/transcoders to a small conv net (e.g. Alex Net) and study it in depth.**
 - Apply sparse dictionary learning to a vision network, potentially a convolutional network such as AlexNet or inceptionv1, thus helping to complete the project initiated by the Distill thread that worked toward completely understanding one seminal network in very fine detail.
 - Despite not being a language model, I think this would meaningfully advance the state of the art and would represent a completion of the project that Olah/Cammarata et al. started years ago.

- Involves developing SAEs for conv nets. The way a convolutional sparse autoencoder could work is:
 - There are two sets of weights as before, the conv-encoder and the conv-decoder.
 - The encoder is a conv net that takes as input a $B \times C \times H \times W$ tensor (as is typical for conv nets). It outputs a $B \times D \times H \times W$ tensor, where D is the number of dictionary elements. The weights of this encoder are therefore $C \times D \times 1 \times 1$ in shape and stride is 1.
 - There is a sparsity penalty placed on the $B \times D \times H \times W$ output of the encoder.
 - The decoder consists of a conv net, with weight shape $D \times C \times 1 \times 1$. We constrain the norm of the weights to be 1 (unclear to me whether it should be the norm of the flattened $D \times C$ matrix or the frob norm of that matrix or if they're equivalent).
 - The network is, as before, trained to reconstruct the input.
- (A transcoder would be similar, except its weights would either be $D \times D \times 1 \times 1$ (in the linear case), or a similar architecture as the conv-SAE in the nonlinear case. We may need to design a type of transcoder that maps features in a layer with one shape to features in a layer with another shape.)
- We can slot the conv-SAEs into every layer of Alexnet after every nonlinearity (Relu and Max pooling). Alexnet is only 5 conv layers and 3 FC layers. Then we'd make visualizations and labels for every feature, and every transcoder connection between features. Twelve years after this network kicked off the deep learning revolution, we'd finally have a full description for how it works.
- **[Lee] Figure out how to build interpretability interfaces for video/other modalities.**
 - One of my favorite elements of [Ellena Reid's project](#)° was that it developed a way to 'visualize' what neurons were activating for in audio samples. Can we improve on this method? Can we do the same for video models? What about other modalities, such as, e.g. [smell](#), or, I don't know, protein structure? Is there a modality-general approach for this?
- **[Lee] Apply SAEs and transcoders to WhisperV2 (i.e. continue [Ellena Reid's work](#)°)**
- **[Lee] Identifying whether or not, in a very small backdoored model, we can detect the backdoor using e.g. e2eSAEs**

- I mostly only care about this to help figure out if it's reasonable to expect mech interp methods to be able to detect backdoors not using the backdoor-ing distribution.
- **[Lee] Interpreting Mamba/SSMs using sparse dictionary learning.**
- **[Lee] Characterizing the geometry of low-level vision SAE features**
 - Features in the lower levels of biological visual systems, conv nets, or vision transformers, tend to have clear structure such as edges, color gradients, etc.
 - There is also an hypothesis that NNs compute not only using sparsely activating features, but also rely on the geometry of these features. Therefore characterizing this geometry in an analysable setting could help reveal insights about how geometry is used for computation. We should first characterize that geometry in an understandable setting, such as visual data.
 - One possible way to analyze the geometry is to identify characteristic features for a set of points on the hypersphere in a low level visual representation space (e.g. by finding the max activating dataset examples for certain SAE dict elements or for random directions in the 1st layer of a convnet/VIT. Then we could map those images/visualizations using a Fourier frequency transform to a space representing the various brightness/color frequencies. These would give us a map of low-level visual space in terms of objective, analyzable quantities instead of just 'pictures'.
 - This has plausibly been done before in biological networks.
 - This characterization might let us study the geometry of the representation space quantitatively rather than relying on qualitative impressions of what the features are. It might also let us construct an 'ideal' low-level visual feature space from the quantitative description by finding what kind of representations the networks appear to be 'trying' to be learning. Speculatively, it might be possible to repeat this procedure in the next layer, but using the representational primitives identified in the first layer.
- **[Lee] Can we understand the first sequence index of a small transformer?**
 - What would it mean to be able to reverse engineer the algorithm learned by a transformer? One way to do this is to take a very simple case and see how it looks. If we can't do it in a very simple case, then we don't have much hope to do it in more complicated cases.
 - One simple case is simply to try to understand what a transformer does mechanistically on the very first timestep of a forward pass that is not a BOS token. (We may be able to treat the BOS activations as fixed biases)

- We are now able to list all the sparse features of a small neural network. Conditioned on any (or the most common) first non-BOS tokens, how does the network make its predictions of the second token?
- This should be relatively simple and may reveal pain points for understanding longer sequences.
- The natural follow up of this project would be to look at the first two tokens instead of just the first.
- **[Lucius] Attempt to understand a toy LM [completely](#)**
 - How much of the performance of small LMs like Tinystories do we currently understand completely? I.e., if we write Python code that implements every n-gram, simple induction head and other heuristic that we can locate in the weights of a Tinystories LM, how well does that code perform compared to the original model?
- **[Stefan] Understand a small model (e.g. TinyStories-2L or a small [TinyModel](#) variant) from start to end, from first to last layer.**
 - Currently we focus on analyzing individual layers (SAEs) or individual Transformer blocks (Transcoders), but how are they connected? E.g. which low-layer Transcoder features connect to which next-layer Transcoder features? Currently decompositions in adjacent layers use different bases, while we can't use the exact same basis (Transcoder inputs likely won't be to single features of previous layers), can we at least get sparse interactions between adjacent transcoder features?

Intrinsic interpretability

- **[Lee] Can we train a small bilinear transformer on either a toy or real dataset, perform sparse dictionary learning on its activations, and understand the role of each sparse dictionary feature in terms of the closed form solution?**
 - There exists an analytical expression for transformers that use bilinear layers (with no layer norm) ([Sharkey 2023](#)). Can we train a small bilinear transformer on either a toy or real dataset, perform sparse dictionary learning on its activations, and understand the role of each sparse dictionary feature in terms of the closed form solution? This may help in identifying fundamental structures within transformers in a similar way that induction heads were discovered.

- **[Lee] Interpretable inference: Can we convert already-trained models into forms that are much easier to completely interpret at little performance cost?**
 - Potentially using e.g.
 - Using [1-bit weights](#) or [MatMul-free operations](#) since both of these might be sparse by default;
 - Using [many small mixture of experts](#) and MOEification.
 - Replacing any SwiGLU layers with bilinear layers
 - Mamba/SSM models, since they must compress the state into a relatively small vector instead of using the whole KV cache as the state, as in attention. A more compressed state might be easier to interpret.
 - [Credit to Chris Mathwin for this idea, who has done some preliminary investigations into it:] Use sparser kinds of attention pattern, as in [entmax-attention](#).
- **[Lee] Develop A Mathematical Framework for Linear Attention Transformer Circuits**
 - Building on A Mathematical Framework for Transformer circuits (Elhage et al. 2022).
 - Elhage et al. 2022 had to freeze the attention pattern on each input, reducing the utility of their formalization for identifying circuits in language models that rely on attention. Linear attention might be a way to simplify the maths further.
 - If this framework also used bilinear MLPs, then it might admit an analytical expression for a somewhat performant transformer. The number of terms may still be exponential in practice, but it might be possible to ‘throw away’ terms using the weight structure, making some assumptions about the input distribution or the activations at each layer.
 - Similarly, if linear attention doesn’t yield to analysis, then perhaps some other sequence modeling components might, such as SSM blocks.

Understanding features (not SDL)

- **[Lucius] Recovering ‘features’ through direct optimisation for auto-interpretability scores**
 - Some people think the most important quality for a feature is being easily human interpretable. I doubt this is actually a good metric to target. But maybe I’m

wrong and it is. So why not try finding features by directly optimizing directions for being interpretable? Example procedure: Have an LLM label all the neurons in another LLM as linear combinations of monosemantic features. Run the labeling multiple times, encouraging re-use of the same set of features for labeling all neurons. Then, take the coefficients for a particular feature for every neuron in a layer and concatenate them together into a vector. Define this to be the dictionary direction of this feature in the layer. Do this for all features. Now, feed the resulting features back to the LLM for another round of the same labeling procedure, with the features from the first round taking the place of the neurons. Repeat this procedure until you end up with a set of highly monosemantic features.

Theoretical foundations for interpretability

- [Singular-learning-theory](#)-related
 - **[Lucius] Understanding SLT at finite data/precision**
 - SLT tells us how to think about exact degeneracies in the loss landscapes of neural networks, and model generalization outside the training set in the limit of infinite training data. How do we coherently extend this to approximate degeneracies in the loss landscape of neural networks, and model generalization to the holdout set outside the infinite data limit?
 - Current estimates of the learning coefficient in neural networks often assume that the loss and learning coefficient are the dominant contributions to the posterior even in real networks at finite data. See e.g. Equation 20 [here](#). Is this a good approximation?
 - **[Lucius] Bounding the local learning coefficient (LLC) in real networks**
 - The rank of the null space of a neural network's Hessian can be used to obtain a lower bound for the network's LLC. This is potentially much cheaper than estimating the LLC directly. If this lower bound proved to be large for LLMs, that would indicate that they are not very degenerate. On the other hand, if this lower bound proved to be small for LLMs, this would indicate that there is potential for techniques based on [exploiting degeneracy in the loss landscape](#) to [decompose](#) LLMs into more interpretable parts.

- Including higher-order terms in the Taylor series than the Hessian makes the lower bound for the LLC tighter. But it's unclear how many terms need to be taken into account in practice to get close to the true value of the LLC in real models. How much does the bound change if we include the next leading term in the Taylor series after the Hessian?
- **[Lucius] Understanding the relationship between the [local learning coefficient \(LLC\)](#) and the [behavioral LLC](#)**
 - The LLC of a neural network is defined relative to a training loss. It is a measure of network simplicity that quantifies how many different parameter configurations in a neural network achieve the same training loss. Inconveniently, the LLC is only well-defined at (local) optima of the loss.
 - In contrast, the LLC of the behavioral loss quantifies how many different parameter configurations result in the same network outputs over the training data set. See section 2.2 [here](#). This is a more restrictive definition since different network outputs can yield the same loss. The LLC of the behavioral loss is thus an upper bound for the LLC of the training loss. The LLC of the behavioral loss is well-defined everywhere in the loss landscape, making it potentially more useful for characterizing the complexity of neural networks at every point in training. However, the behavioral LLC is currently less well understood than the LLC. For example, it is less clearly related to network generalization ability (aside from upper bounding the LLC).
 - What is the relationship between the normal LLC and the behavioral LLC in practice? How much larger is the latter compared to the former in practice? Can we relate the behavioral LLC to network generalization? Can the behavioral LLC help with extending SLT from the Bayesian learning paradigm to real network training?

• Other

- **[Lucius] Extending the current framework for [computation in superposition](#) from boolean variables to floating point numbers or real numbers**
- **[Lucius] Bounding the sparsity of LLM representations**
 - Can we use theoretical arguments and empirical measurements to determine how sparsely we would expect features in LLMs to activate without relying on the assumption that techniques like SAEs actually find the correct features?

- **[Lucius] Relating superposition to the loss landscape**
 - Can we relate the theory of computation in superposition to the geometry of the loss landscape? For example, can we find a relationship between the number of elementary boolean logic gates in superposition a network implements, and the learning coefficient of that network? In other words, are networks that implement fewer gates in superposition simpler according to Singular Learning Theory?
 - On the flip side, can we use the loss landscape as a starting point for developing a better, more realistic model of computation in superposition? For example, how would a network need to implement circuits in superposition for it to be true that networks that implement fewer, simpler circuits have a lower learning coefficient?

Meta-research and philosophy

- **[Lee] Write up reviews/short posts on the links between various concepts in comp neuro and mech interp and philosophy of science and mech interp**
 - E.g.
 - Representational geometry in brains and anns
 - Topological data analysis in brains and anns
 - Dynamical systems for analyzing structure in brains in anns
 - What is an explanation? A primer for mech interp practitioners on the philosophy of science of explanations
 - A history of the Human Genome project in the context of mech interp.
 - The philosophical history of mechanistic interpretability
- **[Lee] What is a feature? What terms should we really be using here? What assumptions do these concepts make? Where does it lead when we take these assumptions to their natural conclusions?**
- **[Lucius] Should we expect some or many of the ‘features’ in current neural networks to be **natural latents**?**
 - If we should not expect them to be natural latents under the strict definition of that term, should we expect them to be natural latents under some relaxed definition?

Engineering

- **[Dan] Create a new, high quality tinystories dataset and model suite (credit to Noa Nabeshima for the idea).**
 - The existing [tinystories dataset](#) is very formulaic, small, and has unusual unicode characters in it. Addressing these issues, and training a small model suite on this new dataset, would be very valuable for:
 - Testing out foundational interpretability methods.
 - Having a language model organism that we could fully reverse engineer.
 - Creating a dataset with the latest LMs will be much cheaper and higher quality than what was used in the existing dataset.
 - Noa Nabeshima has done some work [cleaning up](#) the existing tinystories dataset and training a [4-layer model](#) without layernorm on the clean dataset (it also comes with SAEs and transcoders trained on it). Reach out to Noa (noanabeshima@gmail.com) and/or me (dan@apolloresearch.ai) if interested in taking this on. Subsidies for compute credits for dataset generation and model training may be available.

-
1. [^] Papers from our first project [here](#) and [here](#) and from our second project [here](#).

Mentioned in

- 75 Shallow review of technical AI safety, 2024
- 5 Invitation to lead a project at AI Safety Camp (Virtual Edition, 2025)
- 4 SimpleStories: A Better Synthetic Dataset and Tiny Models for Interpretability

7 comments, sorted by top scoring



Neel Nanda 2y ▼ 8 ▲ ✕ 0 ✓

Therefore, many project ideas in that list aren't an up-to-date reflection of what some researchers consider the frontiers of mech interp.

Can confirm, that list is SO out of date and does not represent the current frontiers. Zero offence taken.
Thanks for publishing this list!

[-] **leogao** 2y ▼ 6 ▲ ✕ 0 ✓

Some takes on some of these research questions:

Looking for opposing feature directions in SAEs

I checked a top-k SAE with 256k features and $k=256$ trained on GPT-4 and found only 286 features that had any other feature with cosine similarity < -0.9 , and 1314 with cosine sim < -0.7 .

SAE/Transcoder activation shuffling

I'm confident that when learning rate and batch size are tuned properly, not shuffling eventually converges to the same thing as shuffling. The right way to frame this imo is the efficiency loss from not shuffling, which from preliminary experiments+intuition I'd guess is probably substantial.

How much does initializing the encoder to be the transpose of the decoder (as done so here and here) help for SAEs and transcoders?

It helps tremendously for SAEs by very substantially reducing dead latents; see appendix C.1 in our paper.



[-] **Logan Riggs** 2y ▼ 1 ▲ ✕ 0 ✓

Some MLPs or attention layers may implement a simple linear transformation in addition to actual computation.

@Lucius Bushnaq, why would MLPs compute linear transformations?

Because two linear transformations can be combined into one linear transformation, why wouldn't downstream MLPs/Attns that rely on this linearly transformed vector just learn the combined function?

[-] **Jason Gross** 2y ▼ 1 ▲ ✕ 0 ✓

[Lucius] Identify better SAE sparsity penalties by reasoning about the distribution of feature activations

- In sparse coding, one can derive what prior over encoded variables a particular sparsity penalty [corresponds to](#). E.g. an L1 penalty assumes a Laplacian prior over feature activations, while a $\log(1+a^2)$ would assume a Cauchy prior. Can we figure out what distribution of feature activations over the data we'd expect, and use this to derive a better sparsity penalty that improves SAE quality?

This is very interesting! What prior does $\log(1+|a|)$ correspond to? And what about using $\prod_i (1 + |a_i|)$ instead of $\sum_i \log(1 + |a_i|)$? Does this only hold if we expect feature activations to be independent (rather than, say, mutually exclusive)?

[–] Jason Gross 2y ▼ 1 ▲ ✕ 0 ✓

[Nix] Toy model of feature splitting

- There are at least two explanations for feature splitting I find plausible:
 - Activations exist in higher dimensional manifolds in feature space, feature splitting is a symptom of one higher dimensional mostly-continuous feature being chunked into discrete features at different resolutions.
 - There is a finite number of highly-related discrete features that activate on similar (but not identical) inputs and cause similar (but not identical) output actions. These can be summarized as a single feature with reasonable explained variance, but is better summarized as a collection of “split” features.

These do not sound like different explanations to me. In particular, the distinction between “mostly-continuous but approximated as discrete” and “discrete but very similar” seems ill-formed. All features are in fact discrete (because floating point numbers are discrete) and approximately continuous (because we posit that replacing floats with reals won't change the behavior of the network meaningfully).

As far as toy models go, I'm pretty confident that the max-of-K setup from [Compact Proofs of Model Performance via Mechanistic Interpretability](#)^o will be a decent toy model. If you train SAEs post-unembed (probably also pre-unembed) with width d_{vocab} , you should find one feature for each sequence maximum (roughly). If you train with SAE width $d_{\text{vocab}}^3 n_{\text{ctx}}$, I expect each feature to split into roughly $d_{\text{vocab}}^2 n_{\text{ctx}}$ features corresponding to the choice of query token, largest non-max token, and the number of copies of the maximum token. (How the SAE training data is distributed will change what exact features (principal directions of variation) are important to learn.). I'm quite interested in chatting with anyone working on / interested in this, and I expect my MATS scholar will get to testing this within the next month or two.

Edit: I expect this toy model will also permit exploring:

[Lee] Is there structure in feature splitting?

- Suppose we have a trained SAE with N features. If we apply e.g. NMF or SAEs to these directions are there directions that explain the structure of the splitting? As in, suppose we have a feature for math and a feature for physics. And suppose these split into (among other things)
 - 'topology in a math context'
 - 'topology in a physics context'
 - 'high dimensions in a math context'
 - 'high dimensions in a physics context'

- Is the topology-ifying direction the same for both features? Is the high-dimensionifying direction the same for both features? And if so, why did/didn’t the original SAEs find these directions?

I predict that whether or not the SAE finds the splitting directions depends on details about how much non-sparsity is penalized and how wide the SAE is. Given enough capacity, the SAE benefits (sparsity-wise) from replacing the (topology, math, physics) features with (topology-in-math, topology-in-physics), because split features activate more sparsely. Conversely, if the sparsity penalty is strong enough and there is not enough capacity to split, the loss recovered from having a topology feature at all (on top of the math/physics feature) may not outweigh the cost in sparsity.

[-]

Mateusz Bagiński

2y

▼

1

▲

✕

0

✓

recently^[1].

empty footnote

[-]

Lee Sharkey

2y

▼

1

▲

✕

0

✓

Thanks! Fixed now

Moderation Log

More from Lee Sharkey

60	[Linkpost] Interpreting Language Model Par...	Lucius Bushnaq, Dan Braun, Oliver ...	1mo	1
81	Mech interp is not pre-paradigmatic	Lee Sharkey	1y	3
45	Attribution-based parameter decomposition...🔗	Lucius Bushnaq, Dan Braun, Stefan...	1y	5

View more

Curated and popular this week

39	Sympathy for both sides of the egregious misalignment debate	Steven Byrnes	7h	5
26	A Mike’s-Eye View of ARC’s Research	Mikewins	3d	0
19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0